integration

November 3, 2025

0.1 Notebook 4: Integration

[ ]:
```
using Polynomials
using Plots
using LaTeXStrings
using LinearAlgebra

macro mark(bool_expr)
    return :(print($bool_expr ? " " : " "))
end
```

0.1.1 Numerical integration

0.1.2 [Exercise 1] Trapezium rule and Simpson's rule

1. Write a function `composite_trapezoidal(u, a, b, n)` to approximate the integral

$$I := \int_a^b u(x)\,\mathrm{d}x$$

using the composite trapezoidal rule with `n` equidistant points $a = x_1 < x_2 < \cdots < x_{n-1} < x_n = b$. Assume that $n \geq 2$.

[ ]:
```
function composite_trapezoidal(u, a, b, n)
    ### BEGIN SOLUTION
    x = LinRange(a, b, n)
    Δ = x[2] - x[1]
    ux = u.(x)
    return Δ * (ux[1]/2 + sum(ux[2:end-1]) + ux[end]/2)
    ### END SOLUTION
end
```

[ ]:
```
@mark composite_trapezoidal(x -> 5, 1, 2, 100) | 5
@mark composite_trapezoidal(x -> x, 1, 2, 100) | 3/2
@mark composite_trapezoidal(x -> x, 1, 2, 2) | 3/2
@mark composite_trapezoidal(x -> x^2, 0, 1, 2) | 1/2
@mark composite_trapezoidal(x -> x^2, 1, 2, 2) | 5/2
```

2. Write a function `composite_simpson(u, a, b, n)` to approximate the integral I using the composite Simpson's rule based on evaluating `u` at an **odd** number `n` of equidistant points such that $a = x_1 < x_2 < \cdots < x_{n-1} < x_n = b$. Assume that `n` is odd and $n \geq 3$.

Note: `n` here is the number of points where the function `u` is evaluated, and not the number of intervals where Simpson's rule is locally applied.

[ ]:
```
function composite_simpson(u, a, b, n)
    @assert n % 2 == 1 "`n` must be odd"
    ### BEGIN SOLUTION
    x = LinRange(a, b, n)
    Δ = x[2] - x[1]
    ux = u.(x)
    return Δ/3 * sum([ux[1]; ux[end]; 4ux[2:2:end-1]; 2ux[3:2:end-2]])
    ### END SOLUTION
end
```

[ ]:
```
@mark composite_simpson(x -> 1  , 1, 2, 101) ≈ 1
@mark composite_simpson(x -> x  , 1, 2, 101) ≈ 3/2
@mark composite_simpson(x -> x^2, 1, 2, 101) ≈ 7/3
@mark composite_simpson(x -> x^3, 1, 2, 101) ≈ 15/4
@mark composite_simpson(x -> x  , 0, 1, 3) ≈ 1/2
@mark composite_simpson(x -> x^2, 0, 1, 3) ≈ 1/3
@mark composite_simpson(x -> x^3, 0, 1, 3) ≈ 1/4
```

0.1.3 [Exercise 2] Implementing a composite integrator

Milne's integration rule reads

$$\int_{-1}^{1} u(x)\,dx \approx \frac{2}{3}\left(2u\left(-\frac{1}{2}\right) - u(0) + 2u\left(\frac{1}{2}\right)\right)$$

1. Write a function `composite_milne(u, a, b, N)`, which returns an approximation of the integral

$$\int_{a}^{b} u(x)\,dx$$

obtained by partitioning the integration interval $[a, b]$ into N equally large cells, and applying Milne's rule within each cell.

[ ]:
```
function composite_milne(u, a, b, N)
    ### BEGIN SOLUTION
    Δ = (b - a) / N
    x₁ = a .+ Δ/4 .+ Δ*(0:N-1)
    x₂ = a .+ Δ/2 .+ Δ*(0:N-1)
    x₃ = a .+ 3Δ/4 .+ Δ*(0:N-1)
    2Δ/3 * u.(x₁) - Δ/3 * u.(x₂) + 2Δ/3 * u.(x₃) |> sum
    ### END SOLUTION
end
```

[ ]:
```
@mark (abs∘composite_milne)(x -> x, -1, 1, 10) < 1e-13
@mark composite_milne(x -> x, 1, 2, 10) ≈ 3/2
@mark composite_milne(x -> x^2, -1, 1, 1) ≈ 2/3
@mark composite_milne(x -> x^4, -1, 1, 1) ≈ 2/12
```

2. Take $u(x) = \cos(x)$, $a = -1$ and $b = 1$. Plot using `scatter` the evolution of the error, in absolute value, for the values of N given, using a logarithmic scale for both axes.

[ ]:
```
u = x -> cos(x)
a, b = -1 , 1

# Number of intervals
Ns = (round^).(10, LinRange(0, 3, 20))

# Exact value of the integral
I_exact = 2sin(1)

### BEGIN SOLUTION
Is = composite_milne.(u, a, b, Ns)
errors = abs.(Is .- I_exact)
scatter(Ns, errors, label="Integration error")
### END SOLUTION

# Set log scale for both axes
plot!(xscale=:log10, yscale=:log10)
```

3. Estimate the order of convergence with respect to N, i.e. find γ such that

$$|\hat{I}_N - I| \propto \beta N^{-\gamma},$$

where I denotes the exact value of the integral and $\hat{I}_N$ denotes its approbetamation. In order to find β and γ, use the function `Polynomials.fit` to find a linear approximation of the form

$$\log|\hat{I}_N - I| \approx \log(\beta) - \gamma \log(N).$$

If your calculation is correct, the function `N -> *N^(- )` should give a good approximation of the integration error.

[ ]:
```
# Calculate  and
### BEGIN SOLUTION
p = fit(log.(Ns), log.(errors), 1)
  = round(exp(p[0]), sigdigits=3)
  = -round(p[1], sigdigits=3)
### END SOLUTION
plot!(N -> *N^(- ), label=L"%$ \times N^{%$ }")
```

[ ]:
```
@mark round( , sigdigits=1)  .1
@mark round( , sigdigits=1)  1e-3
@mark round( , sigdigits=1) == 4
```

0.1.4 [Exercise 3] Composite Gauss-Legendre integration

1. Write a function `legendre(n)` that returns the Legendre polynomial of degree n, in the form of a `Polynomial` structure from the `Polynomials` library. To do this, you can use the

`Polynomials` library and Rodrigues' formula:

$$L_n(x) = \frac{1}{2^n n!} \frac{\mathrm{d}^n}{\mathrm{d}x^n} \left(x^2 - 1\right)^n .$$

Hint (click to show)

- The function `factorial(n)` can be used to calculate the factorial of `n`.
- The function `Polynomials.Polynomial` allows you to create a polynomial from its coefficients:

  ```
  p = Polynomial([1, 2, 3])  # p(x) = 1 + 2x + 3x²
  ```

- The function `Polynomials.derivative` allows you to compute the derivatives of a polynomial:

  ```
  dp = derivative(p)  # dp(x) = 2 + 6x
  ddp = derivative(p, 2)  # ddp(x) = 6
  ```

[ ]:
```
function legendre(n)
    ### BEGIN SOLUTION
    p = Polynomial([-1, 0, 1])
    return 1 / (2^n * factorial(n)) * derivative(p^n, n)
    ### END SOLUTION
end;
```

2. Write a function `get_nodes_and_weights(n)` that computes, without using any libraries other than those imported at the beginning of the notebook, the nodes $(x_i)_{i\in\{1,\dots,n\}}$ and weights $(w_i)_{i\in\{1,\dots,n\}}$ of the Gauss-Legendre quadrature with n nodes. Recall that the nodes and weights should be such that the approximation

$$\int_{-1}^{1} f(x)\,\mathrm{d}x \approx \sum_{i=1}^{n} w_i f(x_i)$$

is exact for any polynomial f of degree up to $2n-1$.
Hint (click to show)

- Recall that the integration nodes are given by the roots of the Legendre polynomial of degree `n`. These roots can be computed using the `roots` function from the `Polynomials.jl` library.
- To construct the Lagrange polynomials to compute the weights, it may be useful to use the `fromroots` and `integrate` functions from the `Polynomials.jl` library.

  ```
  p = fromroots([1., 2.])  # Constructs (x - 1)(x - 2) = x² - 3x + 2
  q = integrate(p)  # q = x^3/3 - 3x^2/2 + 2x
  ```

[ ]:
```
function get_nodes_and_weights(n)
    ### BEGIN SOLUTION
    nodes = sort(roots(legendre(n)))
    weights = zero(nodes)
    for i in 1:n
          = fromroots(nodes[1:end .!= i])
          =   /  (nodes[i])
```

```
        weights[i] = integrate( , -1, 1)
    end
    ### END SOLUTION
    return nodes, weights
end;
```

[ ]:
```
@mark get_nodes_and_weights(5) |> length == 2
@mark get_nodes_and_weights(5)[1] |> length == 5
@mark get_nodes_and_weights(5)[2] |> length == 5
@mark get_nodes_and_weights(1)[1] ≈ [0.]
@mark get_nodes_and_weights(1)[2] ≈ [2.0]
@mark get_nodes_and_weights(3)[1] .|> legendre(3) |> abs∘sum < 1e-10
@mark get_nodes_and_weights(5)[1] .|> legendre(5) |> abs∘sum < 1e-10
@mark get_nodes_and_weights(5)[2] |> sum ≈ 2
```

3. Write a function `composite_gauss_legendre(u, a, b, n, N)` that returns an approximation of the integral

$$\int_a^b u(x)\,\mathrm{d}x$$

obtained by partitioning the integration interval $[a, b]$ into N subintervals of equal length, and applying the Gauss-Legendre quadrature with n nodes in each subinterval.

[ ]:
```
function composite_gauss_legendre(u, a, b, n, N)
    ### BEGIN SOLUTION
    h = (b-a)/N
    X = LinRange(a, b, N + 1)
    z, w = get_nodes_and_weights(n)
    result = 0.
    for i in 1:N
        nodes = X[i] + h/2 .+ z*h/2
        result += h/2 * w'u.(nodes)
    end
    return result
    ### END SOLUTION
end;
```

[ ]:
```
_short(f, n, N) = composite_gauss_legendre(f, 0, 1, n, N)
for d in 1:9
    @assert _short(x -> x^d, 5, 1) ≈ 1/(d+1)
    @assert _short(x -> x^d, 5, 2) ≈ 1/(d+1)
    @assert _short(x -> x^d, 5, 3) ≈ 1/(d+1)
end
@mark !(_short(x -> x^10, 2, 1)  ≈ 1/11)
@mark !(_short(x -> x^10, 2, 2)  ≈ 1/11)
@mark _short(x -> x^10, 5, 200) ≈ 1/11
@mark _short(x -> exp(x), 5, 200) ≈  - 1
```

4. Consider the special case where $u(x) = \cos(x)$, $a = -1$ and $b = 1$, and define the integration error, viewed as a function of N where n is a fixed parameter, by the formula

$$E_n(N) = |\hat{I}_{n,N} - I_{\text{exact}}|.$$

In this equation, I_{exact} is the exact value of the integral while $\hat{I}_{n,N}$ is its approximation using the composite Gauss-Legendre rule. The task is to

- Estimate, for each value of $n \in \{1, 2, 3\}$, the order of convergence of the composite Gauss-Legendre quadrature with respect to N, that is, to find $\beta = \beta(n)$ such that

$$E_n(N) \propto CN^{-\beta}.$$

- Illustrate on the same graph, using the `Plots.scatter` function, the functions E_1, E_2, E_3, for values of N ranging from 1 to 40. Use logarithmic scales for both axes, and include the convergence order found in the legend, by passing the argument `label="n=$n, =$ "` to the `scatter` function.

[ ]:
```
# Function to integrate
u = x -> cos(x)

# Integration interval
a, b = -1, 1

# Exact value of the integral
I_exact = 2sin(1)

### BEGIN SOLUTION
# Number of nodes
ns = [1, 2, 3]

# Number of cells
N = 1:40

p = plot(title="Convergence of Gauss Legendre quadrature", legend=:bottomleft,
         xticks=([1, 5, 10, 20, 30], ["1", "5", "10", "20", "30"]))
for n in ns
    errors = composite_gauss_legendre.(u, a, b, n, N) .- I_exact
    polyfit = fit(log.(N), log.(abs.(errors)), 1)
      = round(- polyfit[1], digits=2)
    scatter!(N, abs.(errors), label="n=$n, =$ ", scale=:log10)
    xlabel!(L"N")
    ylabel!(L"|I - \widehat I_{n,N}|")
end
p
### END SOLUTION
```

0.1.5 [Exercise 4] Gauss Laguerre integration

Our goal in this exercise is to write a program in order to calculate integrals of the form

$$I[f] := \int_0^\infty f(x)\mathrm{e}^{-x}\,\mathrm{d}x$$

To this end, we will use Laguerre polynomials, which are orthogonal polynomials for the following inner product:

$$\langle f, g\rangle := \int_0^\infty f(x)g(x)\mathrm{e}^{-x}\,\mathrm{d}x$$

These polynomials can be constructed by using the Gram-Schmidt algorithm. 1. Using that Laguerre polynomials satisfy the recurrence relation

$$L_{k+1}(x) = \frac{(2k+1-x)L_k(x) - kL_{k-1}(x)}{k+1}, \qquad L_0(x) = 1, \qquad L_1(x) = 1 - x,$$

we first write a function `laguerre(n)` which returns the Laguerre polynomial of degree n.

[ ]:
```
function laguerre(n)
    if n == 0
        return Polynomial([1])
    elseif n == 1
        return Polynomial([1, -1])
    else
        k = n-1
        x = Polynomial([0, 1])
        return ((2k + 1 - x) * laguerre(k) - k*laguerre(k-1))/(k+1)
    end
end
```

2. Write a function `get_nodes_and_weights(n)` which returns the nodes and weights of the Gauss-Laguerre quadrature with n nodes.
   Hint (click to display)
   - Recall that the nodes of the quadrature are the roots of the Laguerre polynomial of degree n. To find these, use the `roots` function from the `Polynomials` package.
     ```
     p = Polynomial([1, 0, -1])
     r = roots(p)  # r = [-1.0, 1.0]
     ```
   - Once you have found the nodes of the quadrature, the weights can be obtained from the relation
     $$\int_0^\infty q(x)\,\mathrm{e}^{-x}\,\mathrm{d}x = \sum_{i=1}^n w_i q(x_i),$$
     which should hold true for any polynomial q of degree at most $2n-1$. Taking $q = \ell_i$ to be the Lagrange polynomial associated with node i immediately gives that
     $$w_i = \int_0^\infty \ell_i(x)\,\mathrm{e}^{-x}\,\mathrm{d}x, \qquad \ell_i = \prod_{\substack{j=1\\ j\neq i}}^n \frac{x - x_j}{x_i - x_j}.$$
   - In order to construct Lagrange polynomials ℓ_i, you may find it useful to use the `fromroots` function from the `Polynomials` package.

```
r = [-1.0, 1.0]
p = fromroots(r)  # p = Polynomial(-1.0 + 1.0*x^2)
```

Recall also that, for a vector `x`, the expression `x[1:end .!= 5]` returns the vector obtained by removing the fifth element from `x`.

- To calculate the integral of Lagrange polynomials against the exponential weight, recall that

$$\int_0^\infty x^n \mathrm{e}^{-x}\,\mathrm{d}x = n!$$

[ ]:
```
function get_nodes_and_weights(n)
    ### BEGIN SOLUTION
    nodes = roots(laguerre(n))
    weights = zero(nodes)
    for i in 1:n
         = fromroots(nodes[1:end .!= i])
         /=  (nodes[i])
        weights[i] = factorial.(0:n-1)' .coeffs
    end
    return nodes, weights
    ### END SOLUTION
end
```

[ ]:
```
@mark get_nodes_and_weights(5) |> length == 2
@mark get_nodes_and_weights(5)[1] |> length == 5
@mark get_nodes_and_weights(5)[2] |> length == 5
@mark get_nodes_and_weights(1)[1] ≈ [1.0]
@mark get_nodes_and_weights(1)[2] ≈ [1.0]
@mark get_nodes_and_weights(3)[1] .|> laguerre(3) |> abs sum < 1e-10
@mark get_nodes_and_weights(5)[1] .|> laguerre(5) |> abs sum < 1e-10
@mark get_nodes_and_weights(5)[2] |> sum ≈ 1
```

3. Write a function `integrate_laguerre(f, n)`, which returns an approximation of $I[f]$ obtained by Gauss-Laguerre integration with n nodes.

[ ]:
```
function integrate_laguerre(f, n)
    ### BEGIN SOLUTION
    nodes, weights = get_nodes_and_weights(n)
    return f.(nodes)'weights
    ### END SOLUTION
end
```

[ ]:
```
@mark integrate_laguerre(x -> x, 5) ≈ 1
@mark integrate_laguerre(x -> x^2, 5) ≈ 2
@mark integrate_laguerre(x -> x^3, 5) ≈ 6
@mark integrate_laguerre(x -> exp(-x), 15) ≈ 1/2
@mark integrate_laguerre(x -> exp(-2x), 15) ≈ 1/3
```

4. Setting $n = 5$, we calculate numerically that the degree of exactness equals 9.

[ ]:
```
n = 5
for i in 1:9
    correct = integrate_laguerre(x -> x^i, n) ▯ factorial(i)
    println("f = x^$i, Rule exact? ", correct)
    @assert correct
end
```

5. Set $f(x) = \sin(x)$, and plot the integration error as a function of n, using appropriate scales for the `x` and `y` axes.

[ ]:
```
### BEGIN SOLUTION
ns = 1:20
f(x) = sin(x)
I_exact = 1/2
Ih = integrate_laguerre.(f, ns)
plot(ns, abs.(Ih .- I_exact), yscale=:log10, xlabel=L"n", ylabel="Error")
scatter!(ns, abs.(Ih .- I_exact))
### END SOLUTION
```

0.1.6 [Exercise 5] Probabilistic integration

Let B_d denote the d-dimensional unit ball for the Euclidean norm:

$$B_d = \left\{ \mathbf{x} \in \mathbb{R}^d : \|\mathbf{x}\| \leq 1 \right\}.$$

The volume of B_d is defined as the integral of the characteristic function over B_d:

$$\mathrm{vol}(B_d) = \underbrace{\int_{\mathbb{R}} \dots \int_{\mathbb{R}}}_{d \text{ times}} \chi(\mathbf{x})\, dx_1 \dots dx_d, \qquad \chi(\mathbf{x}) := \begin{cases} 1 & \text{if } \mathbf{x} \in B_d \\ 0 & \text{otherwise.} \end{cases}$$

Complete the following tasks: - Write a function `hyperball_volume(dim, n)` that calculates the volume of the unit ball in dimension `dim` using a Monte Carlo approach with `n` samples drawn from an appropriate distribution. Your function should return an estimation of the volume together with the standard deviation of the estimator (which you should estimate from the samples).

[ ]:
```
function hyperball_volume(dim, n)
    ### BEGIN SOLUTION
    number = 0
    for i in 1:n
        x = rand(dim)
        number += norm(x) <= 1
    end
    average = number/n
    var = average*(1-average)
    vol,   = average * 2^dim, sqrt(var/n) * 2^dim
    ### END SOLUTION
    return vol,
end
```

Using the function hyperball_volume, plot the volumes for d going from 1 to 15, together with a 99% confidence interval. See the example solution in Figure 1 with $n = 10^7$. You are allowed to use your knowledge of the fact that $\text{vol}(B_2) = \pi$ and $\text{vol}(B_3) = \frac{4\pi}{3}$, but do not use the general formula for the volume of B_d.

[ ]:
```
n = 10^7

### BEGIN SOLUTION
n_dims = 15
dims, vols, vars = 1:n_dims, zeros(n_dims), zeros(n_dims)
for dim in dims
    vols[dim], vars[dim] = hyperball_volume(dim, n)
end

conf = vars / sqrt(.01)
Plots.scatter(dims, vols, label="Volume estimation")
Plots.plot!(dims, vols, ribbon=conf, fillalpha=0.35, label="99% confidence␣
 ↪interval", xlabel="d")
### END SOLUTION
```